

Final Project Presentation

Machine Learning Classification Project

Student: Nurshirak Karimov

Matricola: 550950

Project Requirements and Deliverables

Project Objectives

Goal: explore the dataset, preprocess it, engineer useful features, train and tune machine learning models, and interpret the best model.

Task type: binary classification with target variable ***target***.

The presentation follows the required stages: **Dataset Exploration, Data Preprocessing, EDA, Feature Engineering, Modeling, Hyperparameter Tuning, and Model Interpretation.**

Deliverable	Meaning	Status
D1.1	Dataset exploration	Included
D2.1-D2.2	Preprocessing and cleaned dataset	Included
D3.1	Exploratory data analysis	Included
D4.1-D4.2	Feature engineering and engineered dataset	Included
D5-D7	Modeling, tuning, interpretation	Included
D8.1	15-minute presentation	Included as editable DOCX

Dataset Overview

Understanding the structure before modeling

The dataset contains **9,000** observations and **11** columns.

Feature groups: **8** numerical features, **2** categorical features, and **1** binary target variable.

Numerical features are named *feature_1* to *feature_8*.

Categorical variables are *category_1* and *category_2*.

The target variable is *target*, with classes **0** and **1**.

Dataset property	Value
Rows	9,000
Columns	11
Numerical features	feature_1, feature_2, feature_3, feature_4, feature_5, feature_6, feature_7, feature_8
Categorical features	category_1, category_2
Target	target

Target Distribution and Missing Values

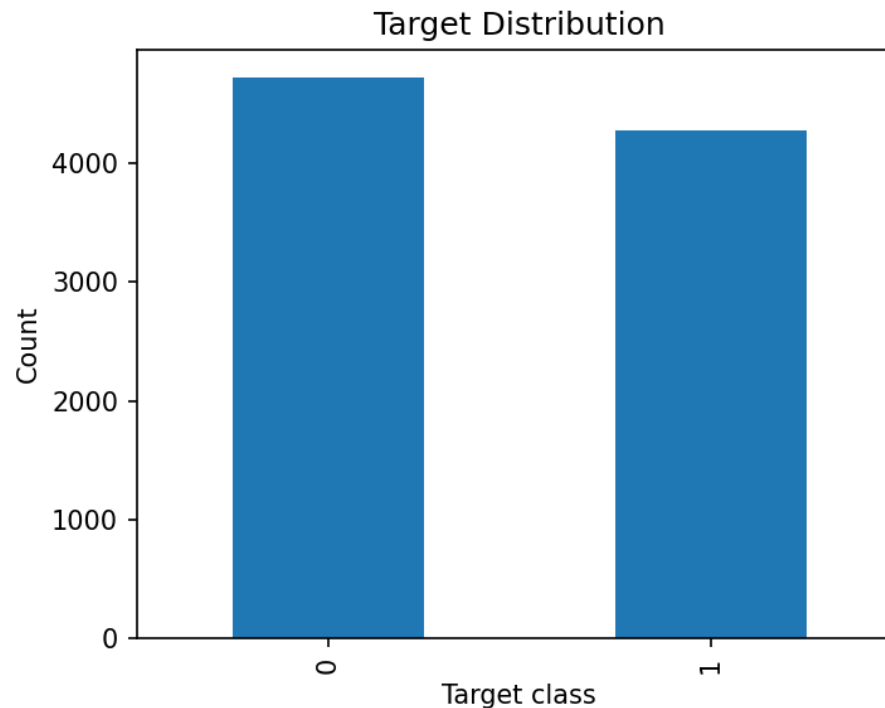
Checking class balance and data completeness

Target **class 0**: 4,721 observations, 52.46%.

Target **class 1**: 4,279 observations, 47.54%.

The target is reasonably balanced, so accuracy is informative, but ROC-AUC, F1, precision, and recall are still used.

Missing values appear only in *feature_6* (**500** values, 5.56%) and *feature_3* (**400** values, 4.44%).



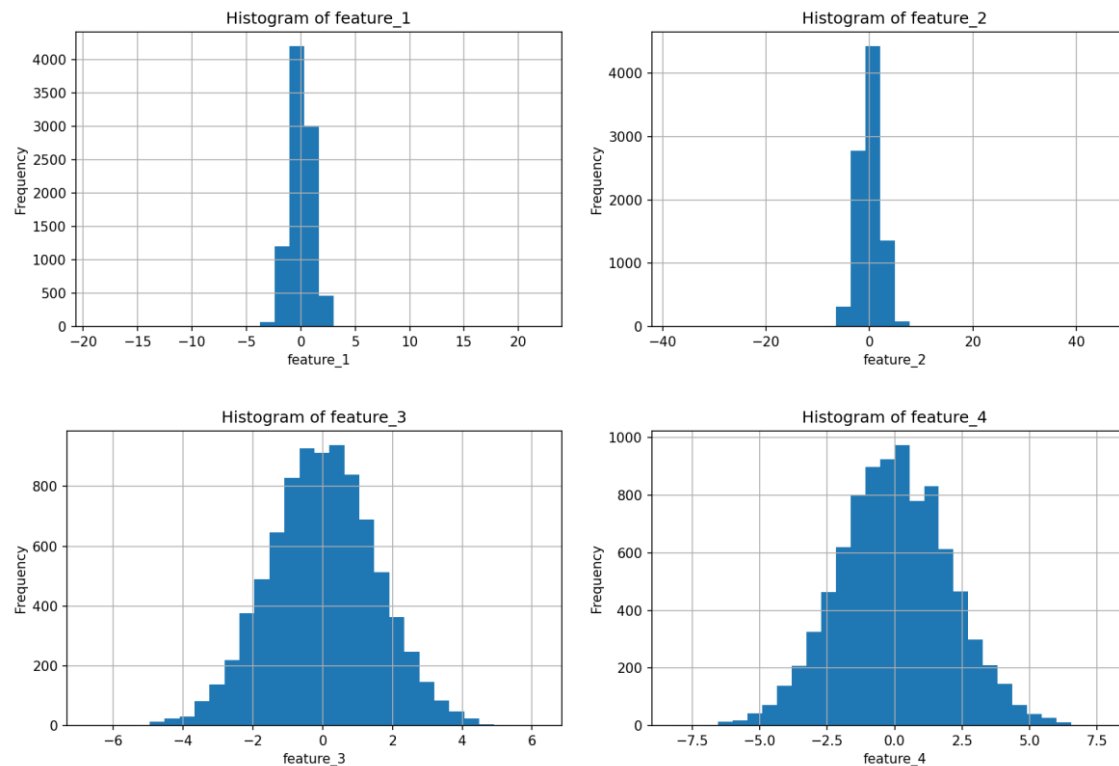
Numerical Feature Distributions I

Histograms for feature_1 to feature_4

Histograms show the **shape**, **spread**, **skewness**, and **concentration** of *numerical values*.

feature_1 to *feature_4* are central to the project because they later show **strong relationships** with the **target**.

The distributions also indicate that outlier handling should be considered.

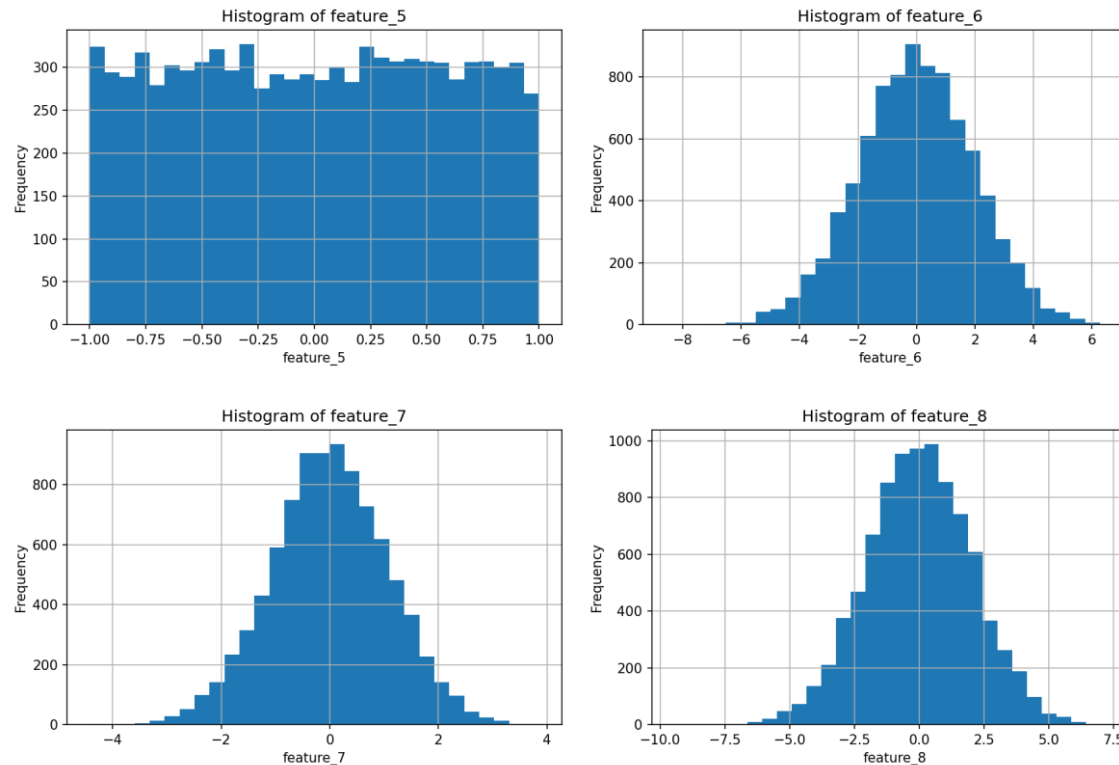


Numerical Feature Distributions II

Histograms for feature_5 to feature_8

The remaining numerical variables show weaker direct relationships with the **target**.

feature_5 to *feature_8* still remain in the dataset because they may interact with other predictors. Keeping them allows the models to test whether they contribute useful multivariate information.



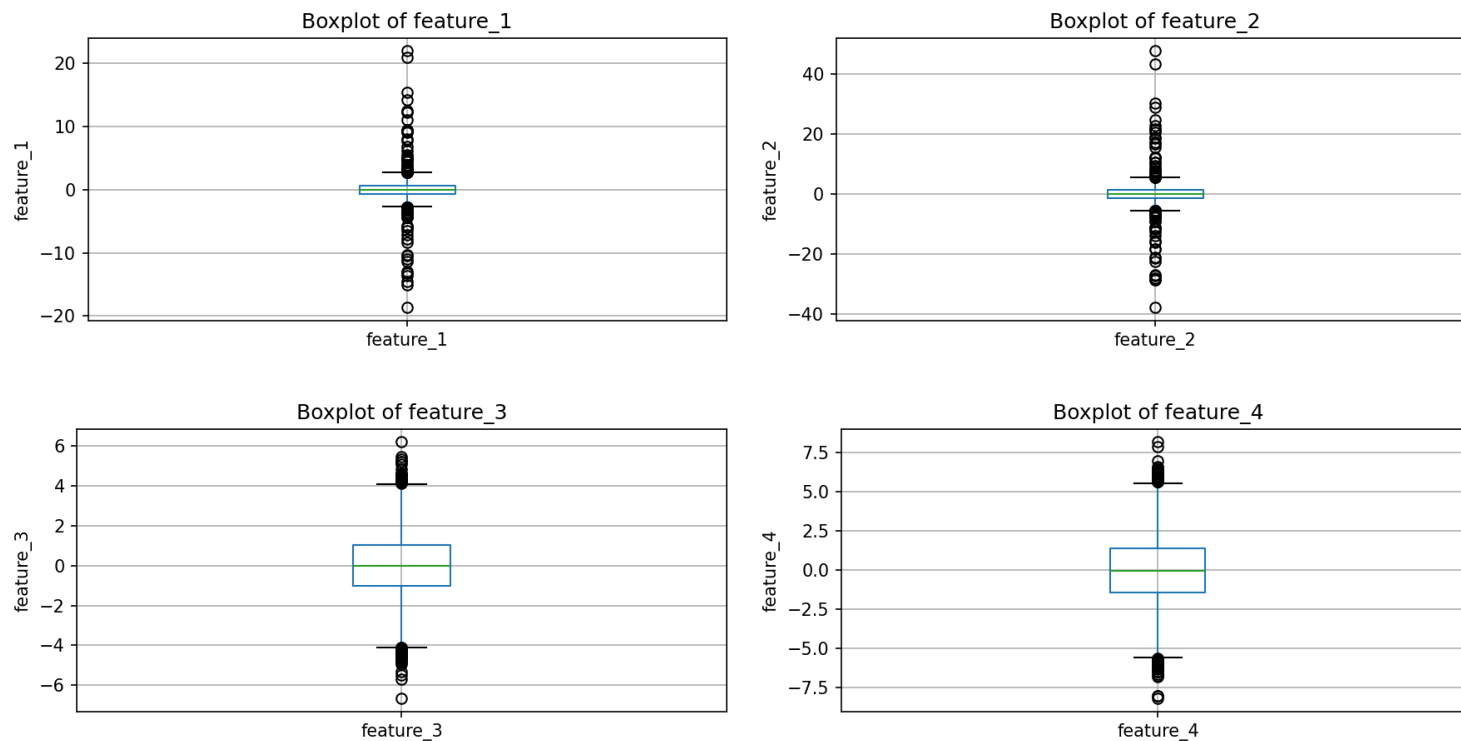
Initial Outlier Diagnostics I

Boxplots for feature_1 to feature_4

Boxplots show **median**, **interquartile range**, **spread**, and **potential outliers**.

feature_1 to *feature_4* contain visible extreme values.

The **IQR rule** was therefore used to detect and clip outliers instead of removing observations.



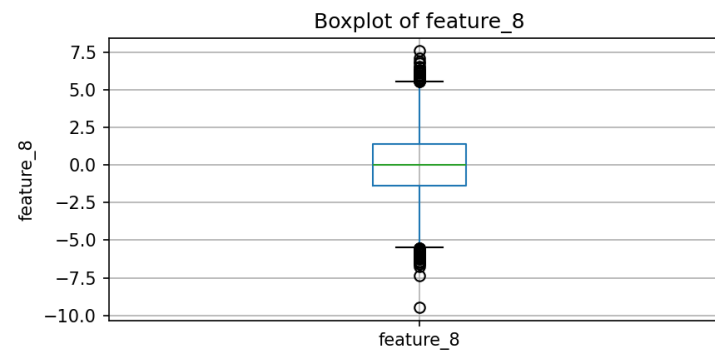
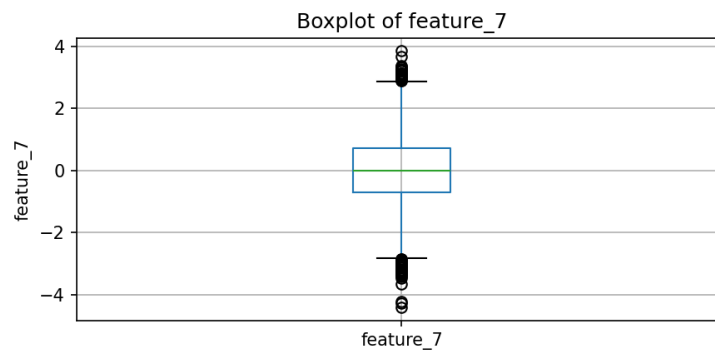
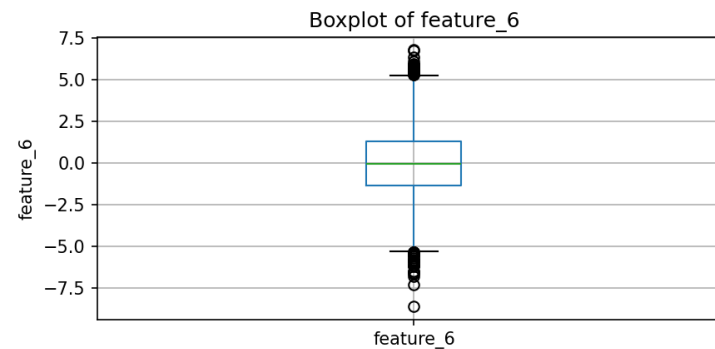
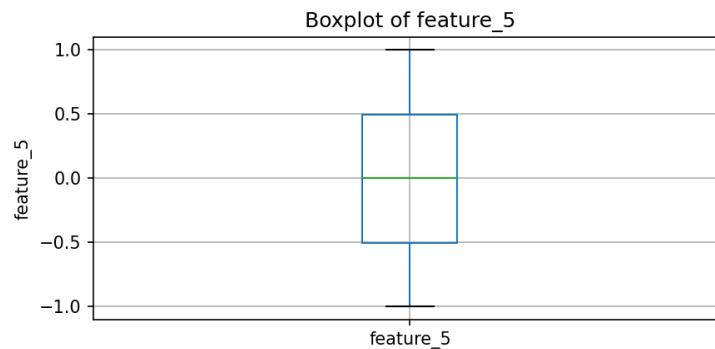
Initial Outlier Diagnostics II

Boxplots for feature_5 to feature_8

feature_5 has no IQR-detected outliers in the initial summary.

feature_6, *feature_7*, and *feature_8* contain moderate numbers of outliers.

The same **preprocessing** rule is applied consistently to all numerical variables.



Initial Correlation Matrix

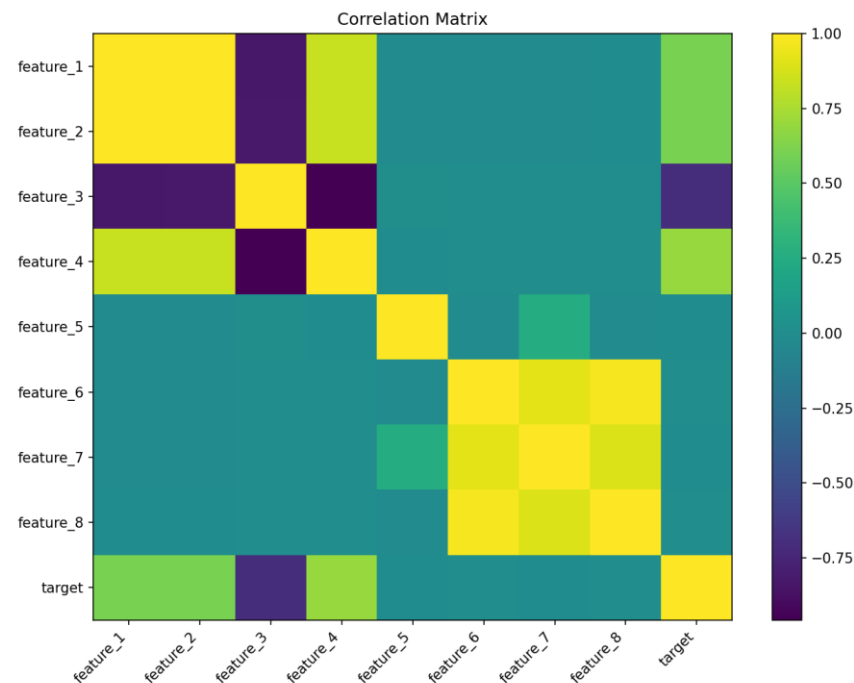
First look at numerical relationships

The **correlation matrix** helps identify linear relationships between numerical features and target.

feature_1, *feature_2*, and *feature_4* show **strong positive association with target**.

feature_3 shows a **strong negative association with target**.

Several numerical predictors are also **highly correlated** with each other, which suggests overlapping information.



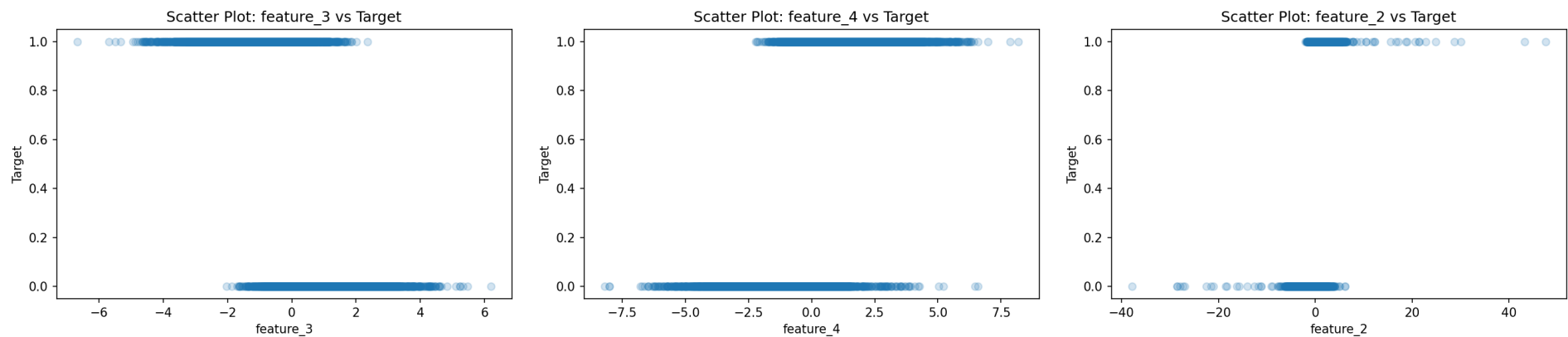
Scatter Plots: Strongest Feature-Target Relations I

feature_3, feature_4, feature_2

Scatter plots visually show how numerical feature values relate to the **binary target**.

Because the target is binary, points appear around two horizontal levels: **class 0** and **class 1**.

The plots help check whether higher or lower values of a feature are more often **associated** with one **target** class.



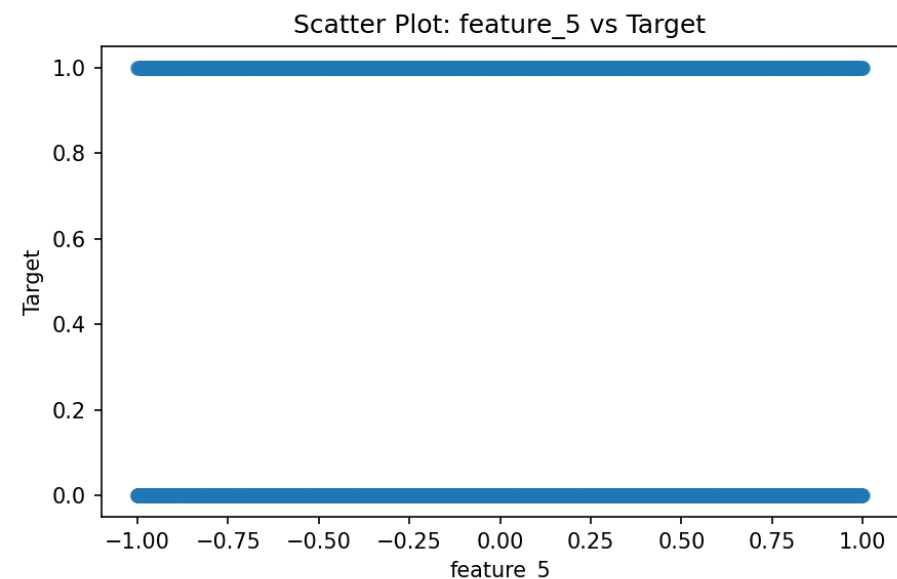
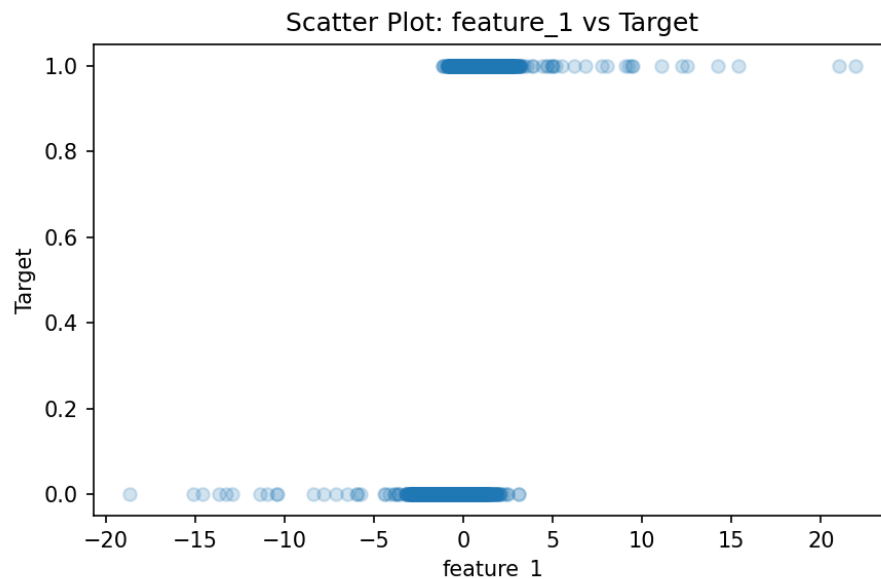
Scatter Plots: Strongest Feature-Target Relations II

feature_1 and feature_5

feature_1 also shows a **visible relationship** with the target.

feature_5 is included among the inspected features but shows much weaker target separation.

This contrast helps distinguish **strong predictors** from weaker ones before modeling.



Data Preprocessing Strategy

Cleaning and preparing data for machine learning

Numerical **missing** values were imputed with the **median** because the **median** is **robust** to skewed distributions and outliers.

Outliers were treated through **IQR clipping** instead of deletion, preserving all observations.

Categorical features were imputed with the **most frequent value** and encoded with **one-hot encoding**.

Numerical features were **scaled** with **StandardScaler** inside the pipeline.

All preprocessing was placed inside a **ColumnTransformer** to keep the workflow **reproducible** and avoid data leakage.

Preprocessing task	Method
Numerical missing values	Median imputation
Outliers	IQR clipping
Categorical missing values	Most frequent imputation
Categorical encoding	One-hot encoding
Scaling	StandardScaler for numerical features

EDA Heatmap After Cleaning

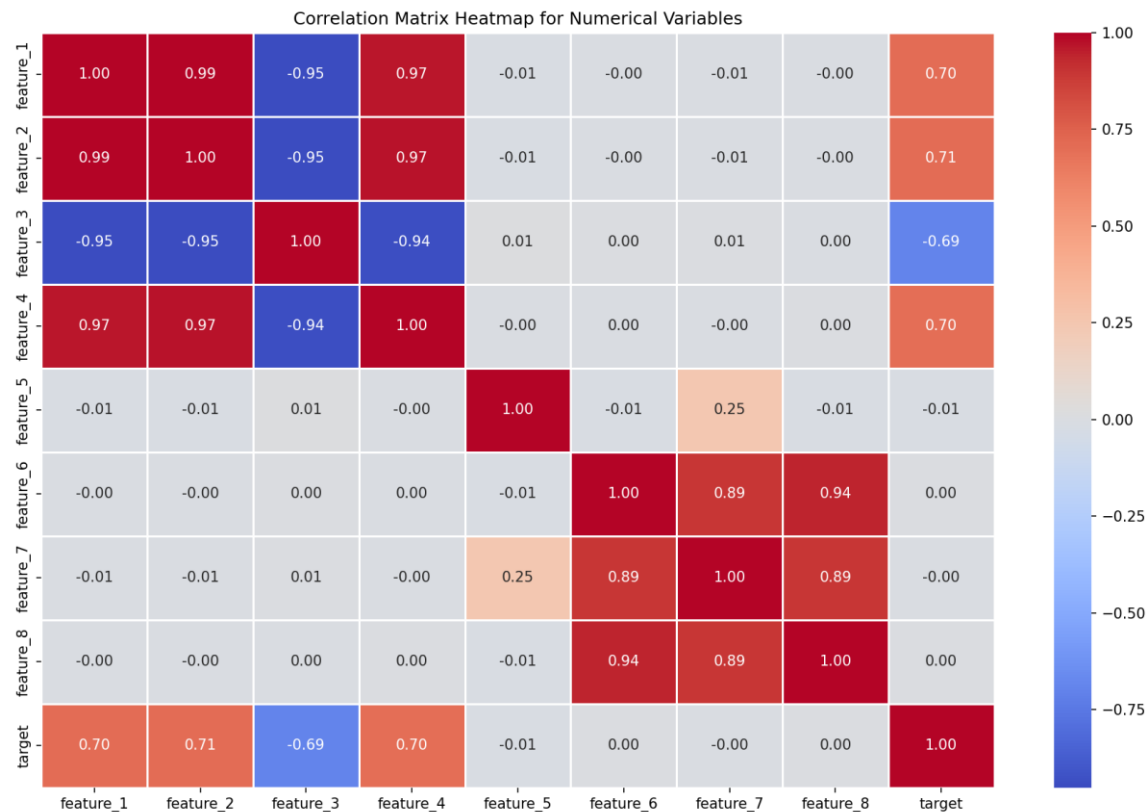
Correlation heatmap for numerical variables

The **heatmap** confirms the strong **relationships** identified earlier.

Strong **target correlations**: *feature_2*, *feature_1*, *feature_4*, and *feature_3*.

feature_1, *feature_2*, *feature_3*, and *feature_4* are also strongly **correlated** with each other.

This suggests that several **predictors** contain overlapping information.



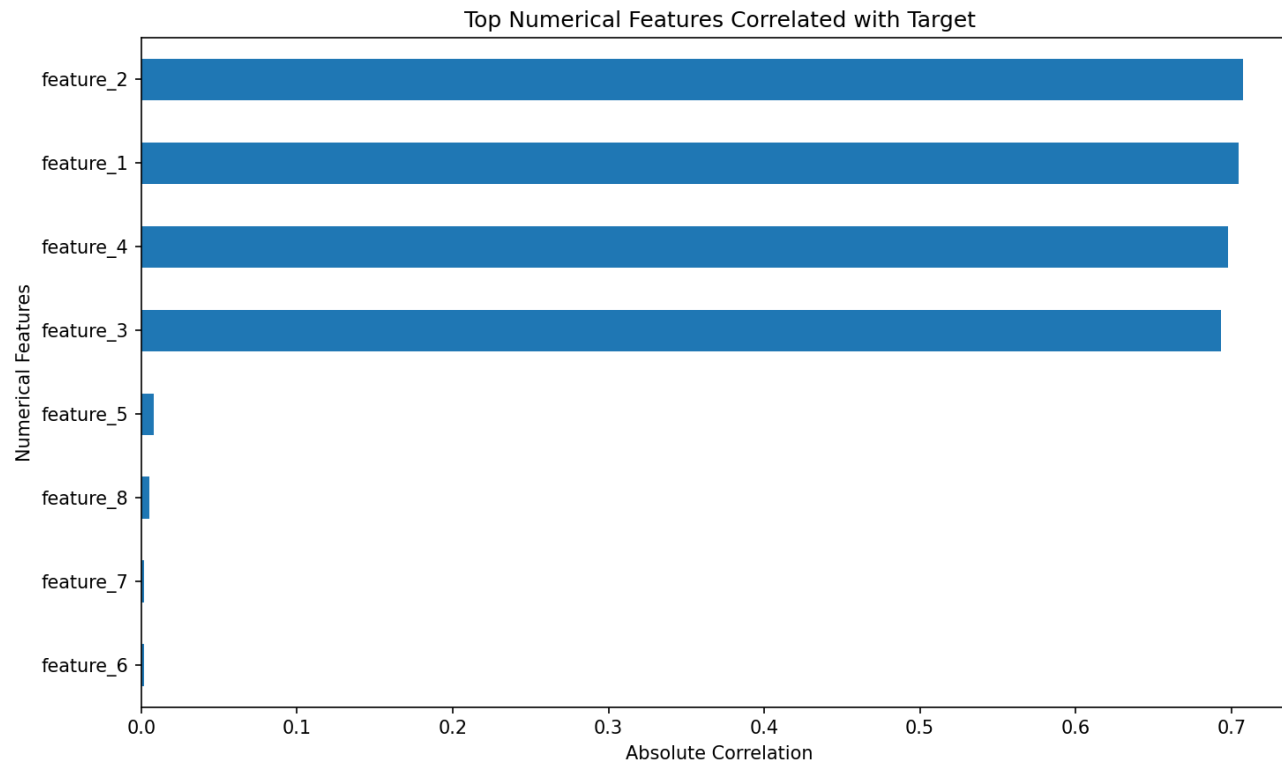
Top Numerical Correlations With Target

Absolute correlation ranking

The strongest absolute **correlations** are: *feature_2* (0.707), *feature_1* (0.704), *feature_4* (0.698), and *feature_3* (-0.693).

feature_3 is negatively **correlated** with the **target**, while *feature_1*, *feature_2*, and *feature_4* are positively **correlated**.

The remaining numerical variables have near-zero direct correlation with **target**.



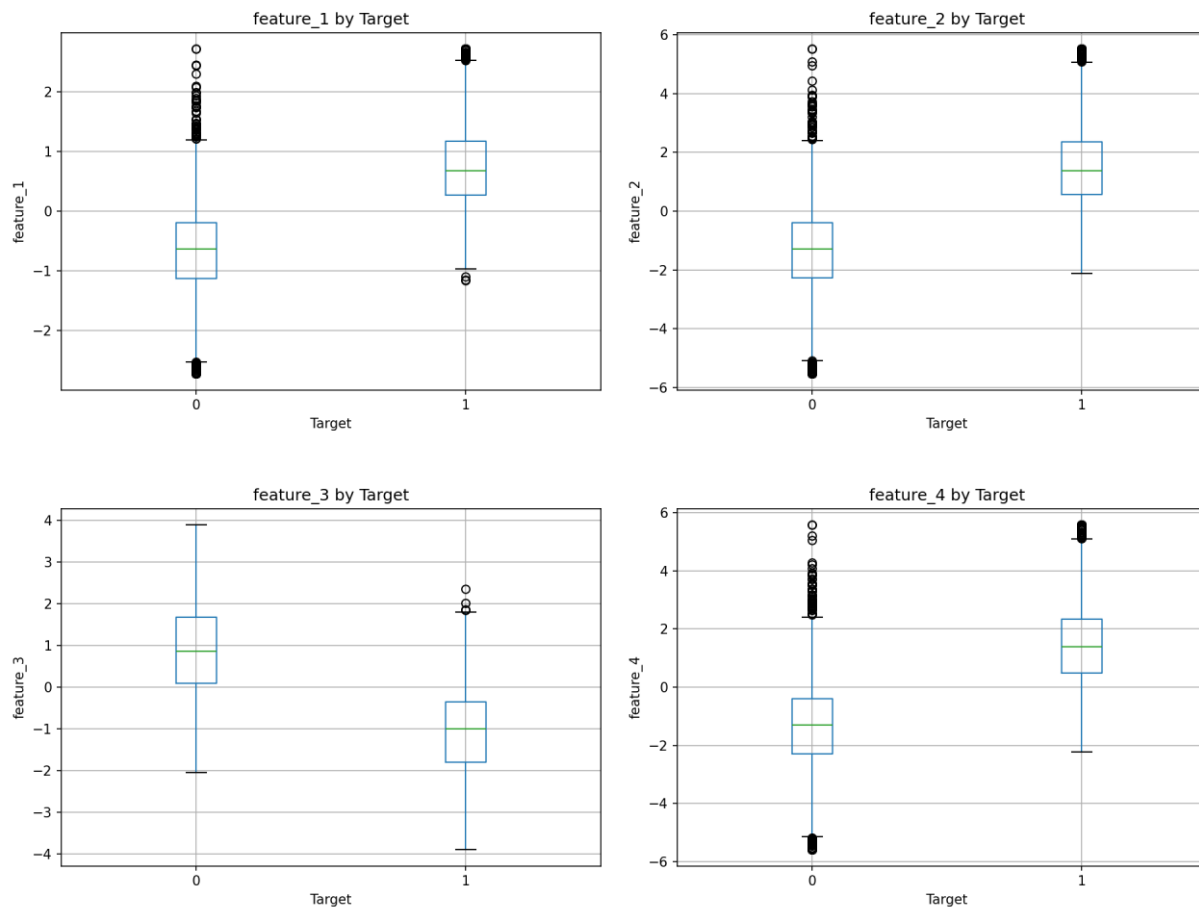
Numerical Features by Target Class I

Boxplots for feature_1 to feature_4

Boxplots by target show how **distributions** differ between classes.

feature_1, *feature_2*, and *feature_4* are generally higher for **target class 1**.

feature_3 is generally lower for **target class 1**, which matches its negative correlation.



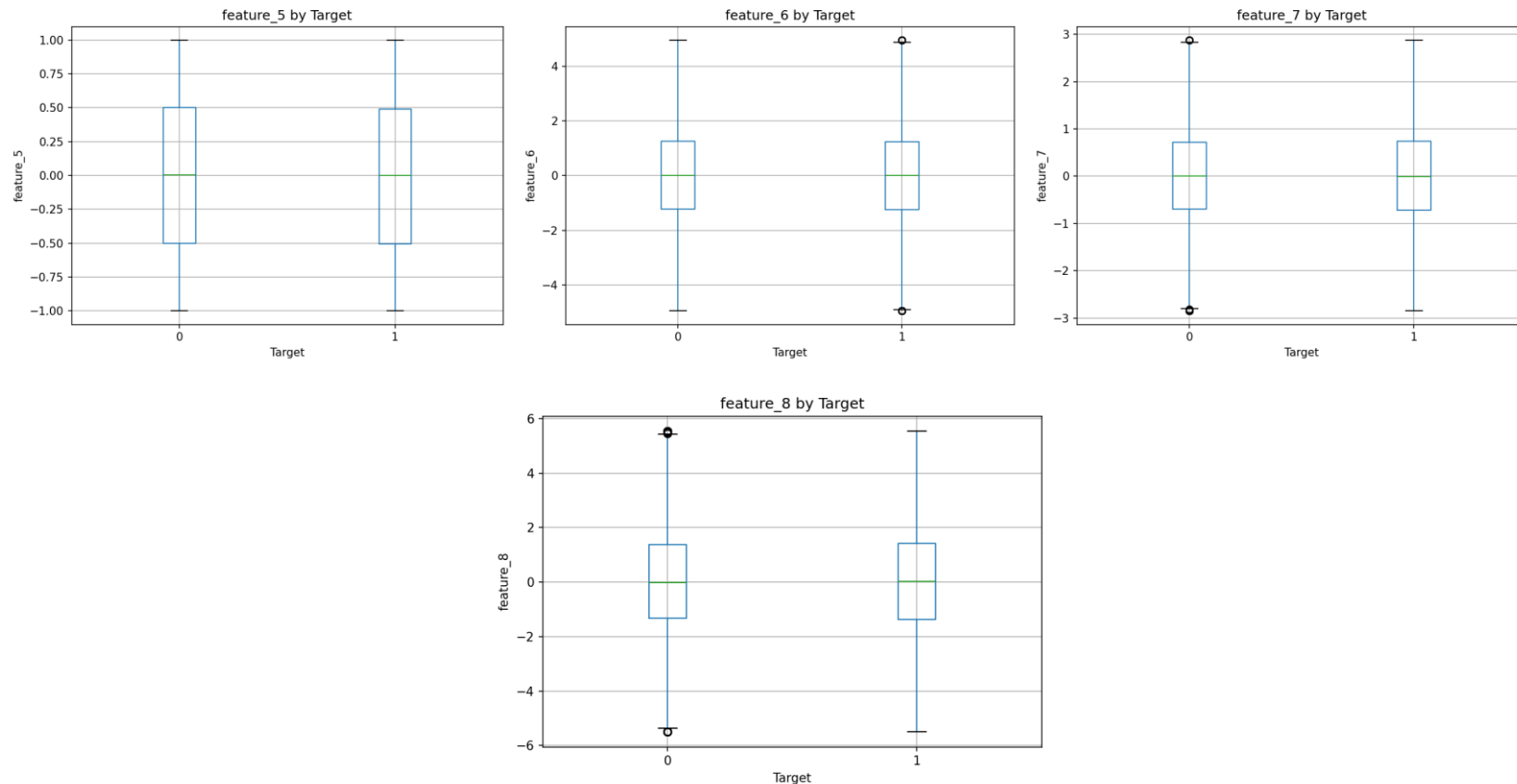
Numerical Features by Target Class II

Boxplots for feature_5 to feature_8

feature_5 to *feature_8* show much weaker separation between target classes.

These variables may still provide limited secondary information in multivariate models.

The plots support the later finding that **the most important model features** are concentrated around *feature_1*, *feature_2*, *feature_3*, and *engineered signals*.



Categorical Feature Analysis

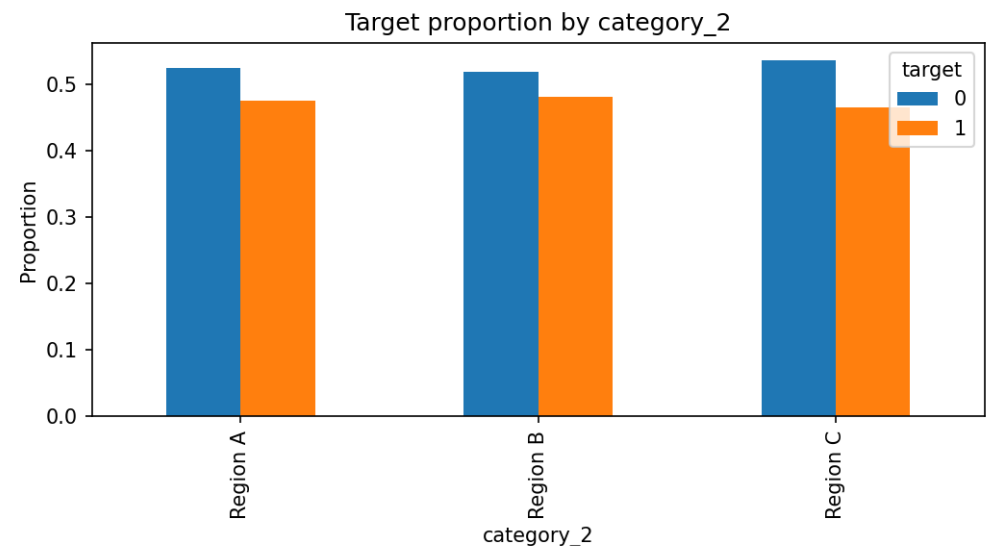
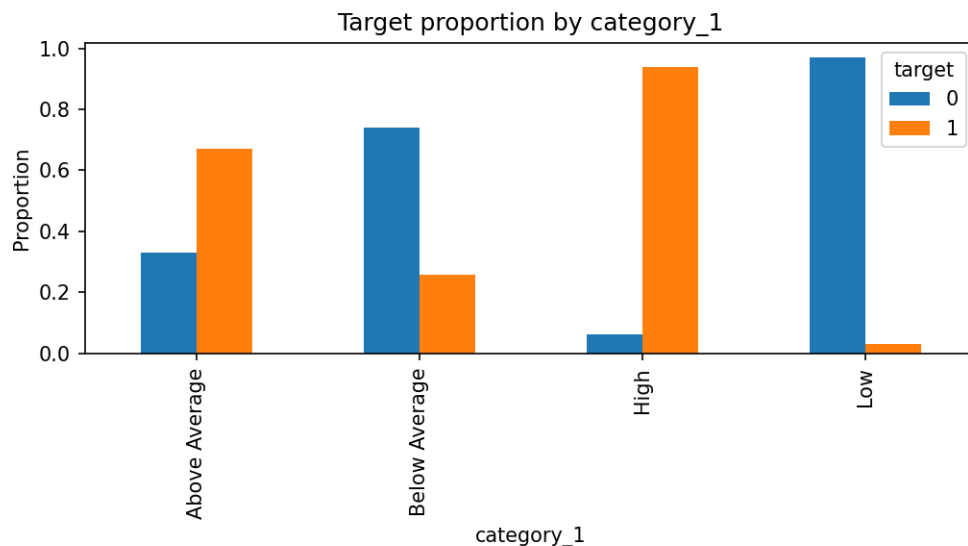
Target proportions by category_1 and category_2

category_1 shows a visible association with **target** during EDA.

category_2 shows weaker separation between target classes.

A **chi-square test** indicates *category_1* is statistically associated with **target**, while *category_2* is not significant at the **0.05** level.

However, later model interpretation shows that *categorical variables* add limited additional **predictive value** once *numerical features* are included.



Statistical Testing and Feature Engineering

From EDA insights to new features

T-tests show statistically significant differences between **target** classes for *feature_1*, *feature_2*, *feature_3*, and *feature_4*.

Chi-square testing confirms that *category_1* is associated with **target**, while *category_2* is not statistically significant.

Engineered features were created from the strongest predictors: **interactions**, **combined signals**, **ratios**, and **overall numerical intensity/variability**.

The goal was to make **important relationships** easier for the models to **capture**.

Feature engineering type	Implementation
Interaction	$\text{feature_1} * \text{feature_2}; \text{feature_3} * \text{feature_4}$
Combined signal	$\text{feature_1} + \text{feature_2} + \text{feature_4}$
Transformation	$\text{abs}(\text{feature_3})$
Ratio	$\text{feature_2} / (\text{abs}(\text{feature_3}) + \text{epsilon})$
Aggregate	overall numerical intensity and variability

Modeling Setup

Training, testing, and evaluation

The engineered dataset was **split** into training and testing sets using an **80/20** split.

Stratification was used to preserve the **target class distribution** in both sets.

Models trained: **Logistic Regression, Decision Tree, Random Forest, Extra Trees, Gradient Boosting, and AdaBoost.**

Evaluation metrics: **accuracy, precision, recall, F1-score, and ROC-AUC.**

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Gradient Boosting	0.886	0.892	0.864	0.878	0.961
Random Forest	0.891	0.900	0.866	0.883	0.958
Logistic Regression	0.893	0.899	0.874	0.886	0.957
Extra Trees	0.888	0.898	0.863	0.880	0.957
AdaBoost	0.889	0.902	0.861	0.881	0.950
Decision Tree	0.841	0.834	0.829	0.832	0.840

Default Model Performance

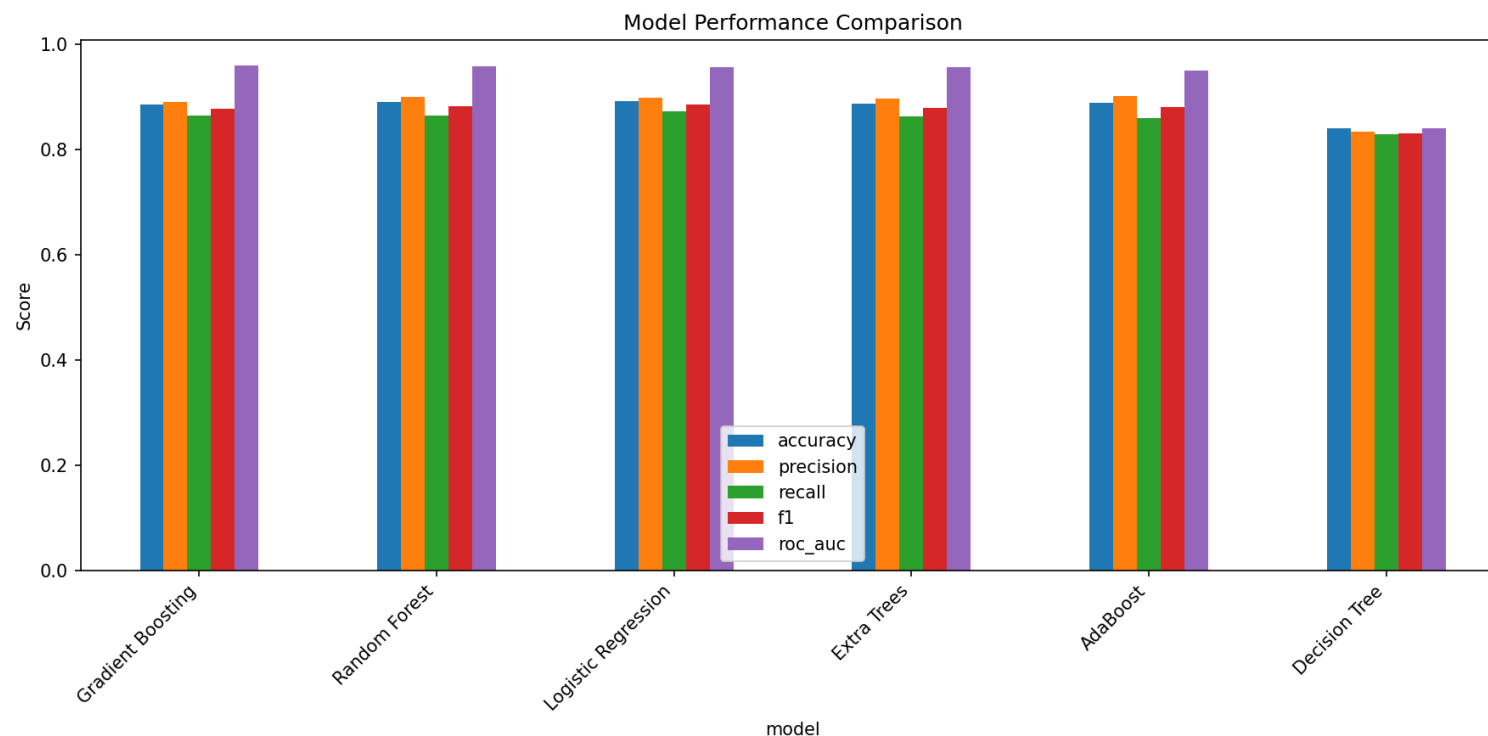
Comparison of all trained models

Gradient Boosting achieved the highest **default ROC-AUC**: 0.9608.

Random Forest and **Logistic Regression** also performed strongly, both above 0.957 ROC-AUC.

Decision Tree performed clearly worse, suggesting that a single tree is less robust than ensemble methods.

The **ensemble models** generally provided stable and high **classification** performance.



Cross-Validation and Hyperparameter Tuning

Making performance estimates more robust

Five-fold cross-validation was used to **check** whether model performance was stable across different training splits.

Gradient Boosting had the highest cross-validated **ROC-AUC mean**: 0.9575.

RandomizedSearchCV was then applied to three **ensemble models**: **Random Forest**, **Extra Trees**, and **Gradient Boosting**.

The **tuning** objective was **ROC-AUC**, with 20 random parameter combinations and 3-fold cross-validation for each model.

Model	CV ROC-AUC mean	CV ROC-AUC std	CV F1 mean
Gradient Boosting	0.9575	0.0025	0.8676
Random Forest	0.9558	0.0036	0.8715
Logistic Regression	0.9534	0.0039	0.8749
Extra Trees	0.9532	0.0026	0.8694
AdaBoost	0.9492	0.0056	0.8647
Decision Tree	0.8377	0.0091	0.8300

Default vs Tuned Ensemble Models

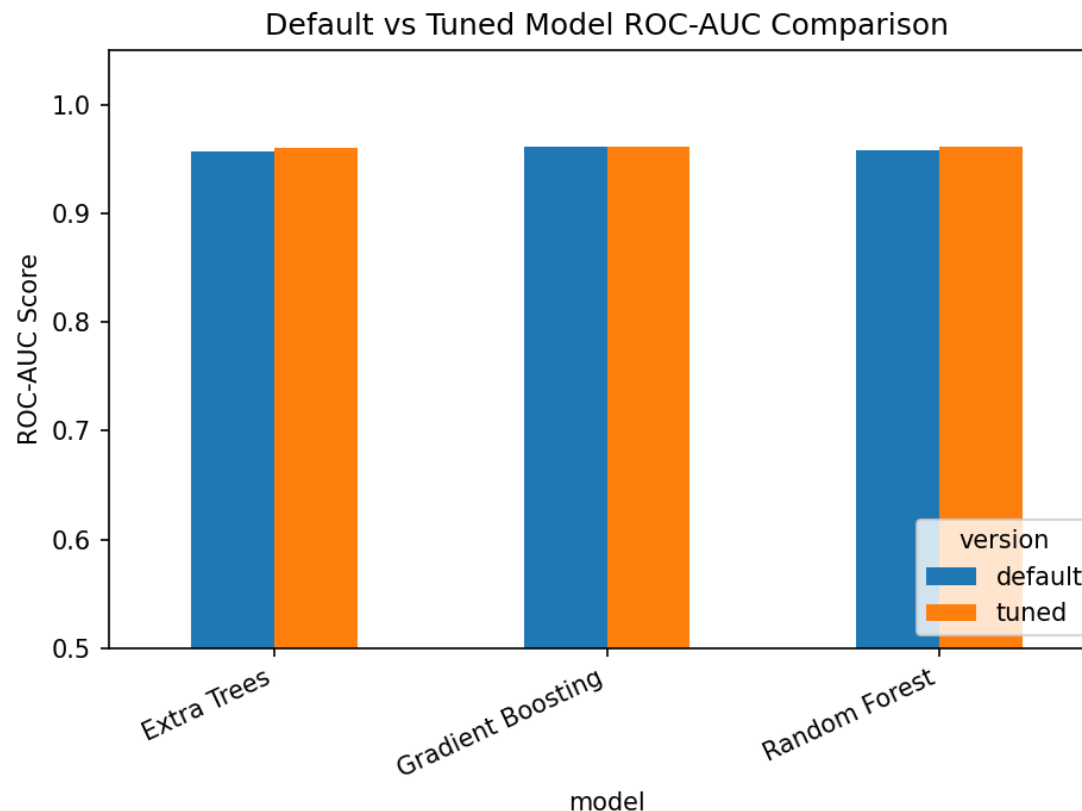
Effect of hyperparameter tuning

After tuning, **Random Forest** achieved the best test **ROC-AUC**: 0.9617.

Gradient Boosting was very close with **ROC-AUC**: 0.9615.

Extra Trees also improved after tuning, reaching **ROC-AUC**: 0.9603.

The **tuned improvements** are modest because the default models were already strong.



Model-Based Feature Importance

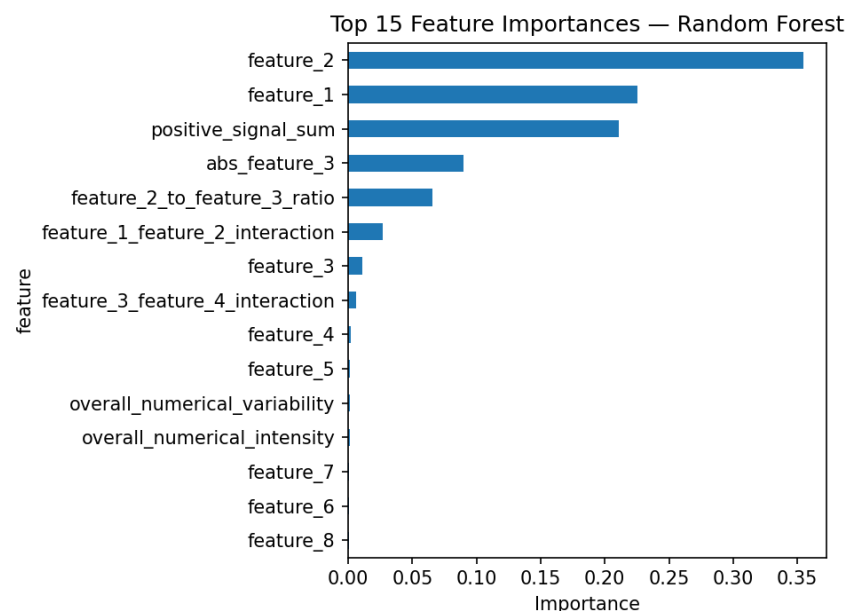
Which variables the best model used most

Model-based **feature importance** identifies which *features* most helped the **tree-based** model **split** the data.

The most important features are *feature_2*, *feature_1*, *positive_signal_sum*, *abs_feature_3*, and *feature_2_to_feature_3_ratio*.

Categorical features receive very low importance, meaning they added little extra information beyond the **numerical predictors**.

Feature importance shows what matters **most**, but not the direction of effect.



SHAP Interpretation

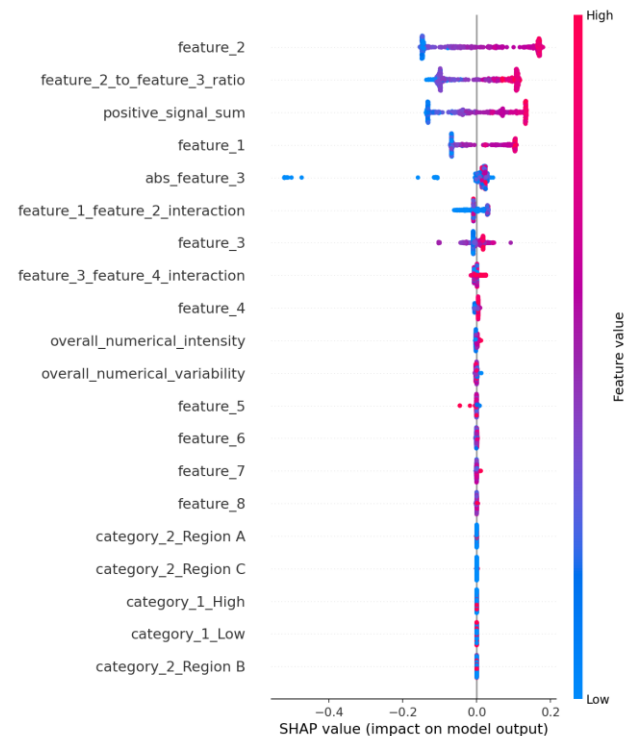
Direction and strength of feature effects

SHAP explains each prediction by assigning **contribution values** to *features*.

Positive SHAP values push the prediction toward target **class 1**; negative SHAP values push it toward **class 0**.

The **SHAP summary** confirms that the **strongest contributions** come from numerical and engineered features.

SHAP complements **feature importance** by showing both importance and direction of influence.



Main Conclusions and Recommendations

Final project summary

The dataset contains a strong **predictive signal**, especially in *feature_1*, *feature_2*, *feature_3*, *feature_4*, and *engineered features* derived from them.

Median imputation, **IQR** clipping, **one-hot encoding**, and **scaling** produced a clean and reproducible modeling **pipeline**.

Ensemble models performed best overall; the **best tuned model** was **Random Forest** with **ROC-AUC 0.9617**.

Model **interpretation** shows that *categorical variables* had limited marginal contribution after accounting for **numerical predictors**.

Recommendation: use the **tuned Random Forest** as the **final predictive model**, while keeping **Gradient Boosting** as a strong alternative benchmark.

Final point	Result
Best tuned model	Random Forest
Best tuned ROC-AUC	0.9617
Most important <i>features</i>	feature_2, feature_1, positive_signal_sum, abs_feature_3
Interpretation method	Model-based importance + SHAP